

UNIVERSIDADE ESTADUAL DE SANTA CRUZ

CIÊNCIA DA COMPUTAÇÃO

CIBELLE SOUSA RODRIGUES

DAVID JÚNIO MARIANO DOS SANTOS

LAISA PEREIRA DA FRANÇA

REDES NEURAIS COM YOLO

ILHÉUS, BA

2025

CIBELLE SOUSA RODRIGUES

DAVID JÚNIO MARIANO DOS SANTOS

LAISA PEREIRA DA FRANÇA

REDES NEURAIS COM YOLO

Atividade avaliativa apresentada como requisito parcial para obtenção de aprovação na disciplina Inteligência Artificial, com código DEC000068, ministrada pelo Professor Otacílio José Pereira, na Universidade Estadual de Santa Cruz, Ilhéus.

ILHÉUS, BA

2025

1. Inspeção de Qualidade em Placas de Circuitos Impressos

Roberto Mendes, atual Gerente de Qualidade da EletroTech, uma fábrica de médio porte especializada no fornecimento de placas eletrônicas para o setor de eletrodomésticos, entrou em contato com os estudantes da UESC ao saber do projeto de iniciação científica em Visão Computacional do curso de Ciência da Computação.

Sua motivação surgiu de um gargalo crítico na linha de montagem da EletroTech. Com a recente expansão da empresa a demanda pelas placas de circuitos impressos (PCBs) disparou, o que fez com que seus inspetores chegassem a verificar visualmente cerca de 2.000 placas diariamente, um processo extremamente exaustivo.

A consequência imediata foi o aumento do custo da produção devido ao descarte excessivo de placas que foram equivocadamente aprovadas na inspeção mesmo contendo defeitos como curtos-circuitos, circuitos abertos, mordida nas bordas e furos faltantes. Além do prejuízo financeiro direto com devoluções e retrabalho, a venda de placas com defeitos aos clientes da empresa vem cada vez mais ameaçando a reputação da marca no mercado.

Não é, entretanto, a primeira vez que Roberto tenta resolver a questão por meio da automatização do processo de inspeção. Recentemente, uma startup de São Paulo vendeu ao Roberto uma solução de reconhecimento de imagens que inicialmente parecia ser muito promissora, mas logo demonstrou-se robustamente falho ao ter que lidar com um novo modelo de placas da EletroTech.

Aos 32 anos e entusiasta da inovação, Roberto resiste à ideia de apenas ampliar o quadro de funcionários e consequentemente elevar significativamente o custo operacional da EletroTech. Ele vê na parceria com os estudantes da UESC uma última oportunidade de validar uma solução tecnológica eficiente antes de recorrer a métodos tradicionais.



Motivados pelo desafio real e pelo impacto na indústria local, os estudantes da UESC propõem o desenvolvimento do Hawkeye. O sistema consiste em uma solução de visão computacional baseada na arquitetura YOLOv8, projetada especificamente para a inspeção visual de PCBs diretamente na esteira de produção.

O objetivo central da aplicação é processar o fluxo de imagens em tempo real e identificar, com alta precisão, as áreas da placa que apresentam anomalias. O sistema desenhará caixas delimitadoras nos monitores de controle, alertando os operadores instantaneamente para a segregação das peças defeituosas, garantindo assim a qualidade final sem reduzir a velocidade da linha.

2. O Treinamento do modelo

Para o treinamento do modelo Hawkeye, a equipe selecionou o dataset público PCB_detection, disponibilizado via Roboflow Universe. Este conjunto de dados foi escolhido por sua especificidade e qualidade, contendo um total de 1.807 imagens de placas de circuito impresso. O dataset já se encontra devidamente anotado com as classes que correspondem exatamente às dores da persona criada para a atividade, abrangendo cinco categorias de detecção: a placa em si (PCB), furos faltantes (missing_hole), falhas na borda (mouse_bite), circuitos abertos (open_circuit) e curtos-circuitos (short).

Em termos de organização para o experimento, os dados já estão segregados em uma divisão estatística robusta: 61% das imagens (1.098) estão destinadas ao treino, 25% (447) para validação durante o aprendizado e 14% (262) reservadas exclusivamente para o teste final. Vale ressaltar que, na versão do dataset escolhida (v3), não foram aplicadas etapas de pré-processamento ou aumento de dados, o que nos fornece um cenário "puro" ideal para analisar o comportamento da rede neural e identificar potenciais ocorrências de overfitting em sua forma nativa.

3. Aspectos Técnicos e Justificativas de Projeto

A arquitetura técnica do experimento foi desenhada para atender aos requisitos de desempenho industrial da EletroTech e, simultaneamente, cumprir o objetivo acadêmico de análise de Underfitting e Overfitting. As decisões técnicas fundamentam-se nos seguintes pilares: YOLOv8, Transfer Learning e Monitoramento de Curvas de Aprendizado.

O primeiro pilar é a escolha da arquitetura YOLOv8. Optamos por este modelo devido à sua reconhecida capacidade de inferência em alta velocidade (baixo tempo de latência). Embora os testes neste trabalho sejam realizados em imagens estáticas do dataset para fins de validação acadêmica, a escolha do YOLO visa provar que a solução teria capacidade técnica de operar em tempo real (acima de 30 FPS) se implantada na esteira da fábrica, garantindo a agilidade necessária para a inspeção visual contínua.

O segundo pilar é a estratégia de Transfer Learning. Ao invés de treinar a rede do zero (o que exigiria um poder computacional imenso e milhões de imagens), utilizaremos os pesos pré-treinados do modelo (como o yolov8n.pt). Isso permite que o sistema aproveite o conhecimento prévio de detecção de formas e bordas, focando o aprendizado apenas nas especificidades das falhas nas placas eletrônicas, acelerando a convergência do treinamento.

Por fim, o terceiro pilar é o Monitoramento de Curvas de Aprendizado. A divisão do dataset em conjuntos de Treino e Validação é a chave para o aspecto técnico do tema. Durante as épocas de treinamento, monitoraremos a divergência entre a perda (loss) de treino e a perda de validação. Caso a perda de treino diminua enquanto a de validação estagne ou suba, teremos a comprovação técnica do Overfitting (o modelo decorou as placas do treino mas não generaliza), o que validará a hipótese de falha do sistema anterior contratado pelo cliente.

4. Análise do modelo e indicadores de avaliação

Para avaliar a eficácia do modelo, selecionamos dois indicadores fundamentais para a indústria: Precisão e Recall. Além disso, apresentamos o mAP como métrica global. A base para todos os cálculos é a Matriz de Confusão gerada durante a validação, que nos mostra onde o modelo acertou e onde ele confundiu as classes.

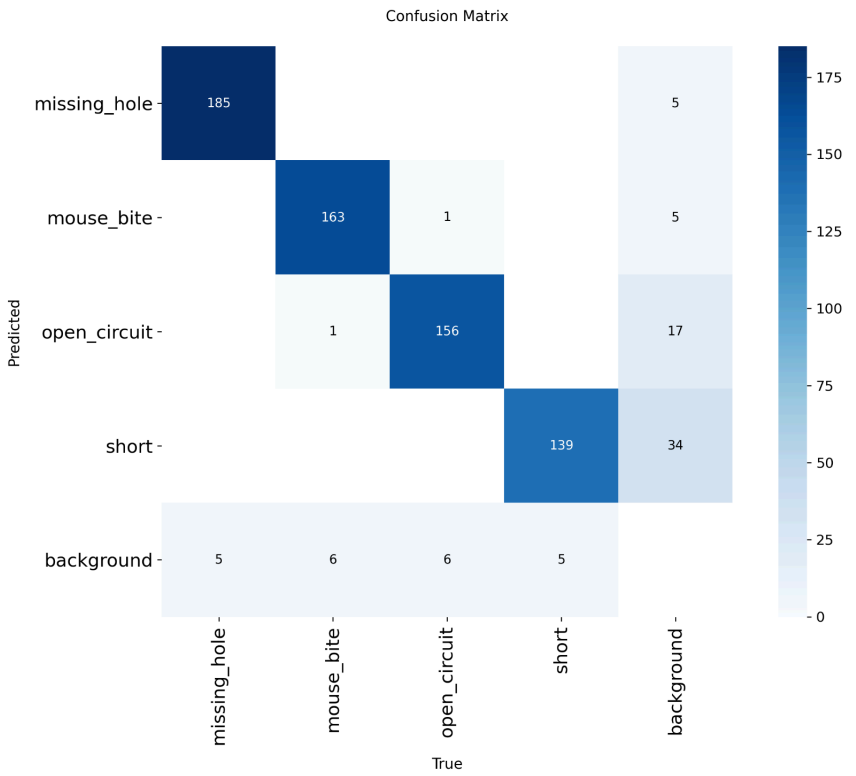
Outrossim, antes de nos aprofundarmos no cálculo precisamos entender o conceito de cada métrica escolhida e a motivação pela escolha de cada uma. Além disso, é importante ter em mente que necessitamos de medidas que condizem com o nosso cenário, ou seja, que traduzam prejuízo financeiro e controle de qualidade, uma vez que estamos lidando com um produto que será destinado ao cliente final.

Precisão: responde à pergunta: "De todas as vezes que o modelo disse que havia um defeito, quantas vezes ele estava certo?". Ela mede a “pureza” das detecções, uma precisão alta significa que o modelo raramente dá "alarmes falsos" (falsos positivos).

- Fórmula: $\frac{VP}{VP+FP}$
- VP (verdadeiro positivo): O modelo acertou o defeito.
- FP (falso positivo): O modelo inventou um defeito onde não tinha.

Justificativa pela escolha: Se a precisão for baixa, o sistema vai parar a esteira o tempo todo alertando sobre defeitos que são apenas poeira ou reflexo de luz (falsos positivos). Cada parada desnecessária custa dinheiro e tempo dos operadores.

Fazendo a conexão com os dados, nota-se que na matriz de confusão gerada, a classe “short” (curto-circuito) teve 34 alarmes falsos (confundiu fundo com curto), baixando a sua precisão. Isso mostra que o modelo ainda tem um custo operacional (média precisão) pois vai exigir que um humano verifique alguns alarmes falsos.

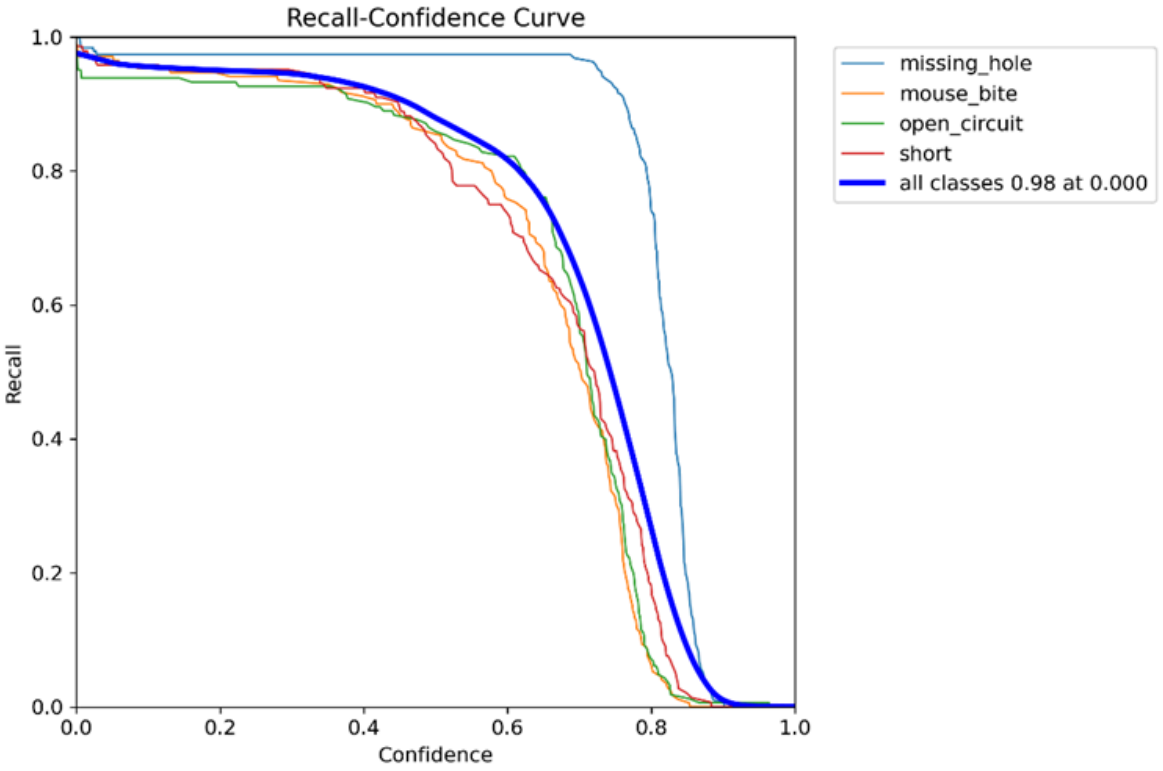


Recall: responde à pergunta: "De todos os defeitos que realmente existiam nas placas, quantos o modelo conseguiu encontrar?". Ele é a medida da “abrangência”, assim um recall alto significa que o modelo raramente deixa passar um defeito (poucos falsos negativos).

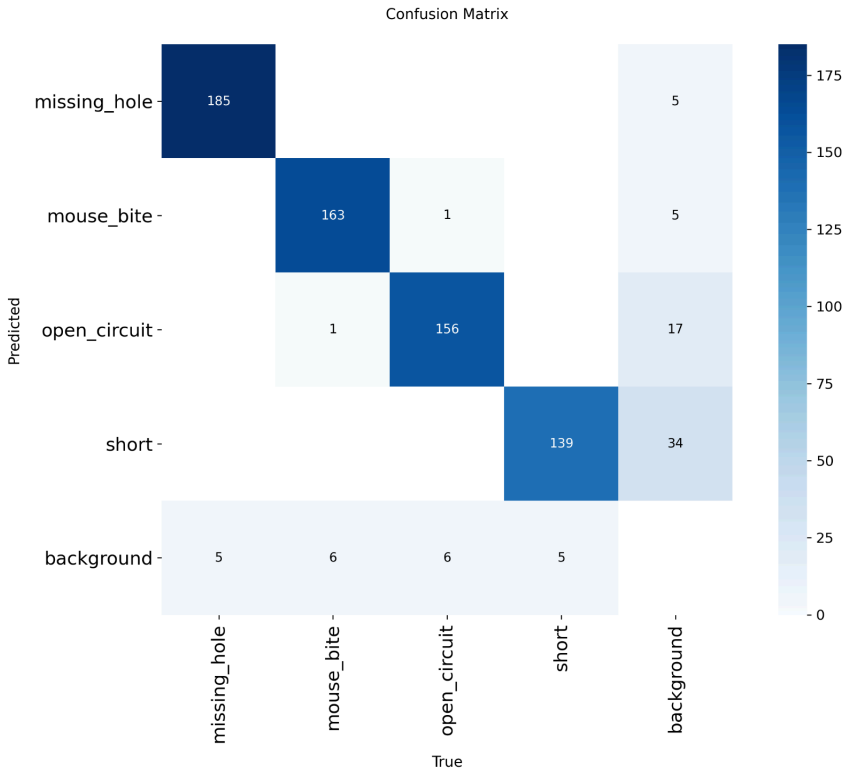
- Fórmula: $\frac{VP}{VP+FN}$
- VP (verdadeiros positivos): O modelo acertou o defeito.
- FN (falsos negativos): O modelo não viu o defeito e disse que estava tudo bem.

Justificativa pela escolha: No contexto do trabalho, se o recall for baixo, significa que o modelo está deixando passar placas defeituosas. Elas vão chegar na casa do consumidor final e causar prejuízo e danos à imagem da marca EletroTech. Dessa forma, visando o controle de qualidade, o recall costuma ser a métrica mais crítica.

Fazendo a conexão com os dados, no gráfico gerado acerca do recall vemos curvas altas, o que é um bom sinal. Se o Recall fosse baixo, significaria que o modelo está sendo "negligente", classificando placas defeituosas como boas (falsos negativos).



Além disso, ao analisar a matriz da confusão na classe missing_hole (buraco faltando). De 190 furos faltando, o modelo achou 185 e perdeu apenas 5, o que significa que ele tende a achar a maioria dos defeitos.



mAP: O mAP vem com a proposta de evitar que o modelo nos engane, uma vez que ao olhar somente para a precisão teríamos uma determinada opinião e ao olharmos somente para o recall, definiremos outra. Note que temos 4 tipos de defeitos: missing_hole, mouse_bite, open_circuit, short. Pode ser que o modelo seja ótimo em achar furos, mas péssimo em achar curto-circuito.

Dessa forma, o mAP (média das precisões médias) pega a nota de cada defeito e tira uma média final, ele nos dá uma nota única para o modelo todo. Em vez de dizer "o modelo é bom nisso, mas ruim naquilo", você diz: "O mAP global é 92%".

Em outras palavras, isso significa que utilizamos o mAP como indicador principal porque Precisão e Recall são métricas antagônicas (geralmente quando uma sobe, a outra desce). O mAP calcula a área sob a curva de compensação entre elas, fornecendo um valor único que garante que o modelo é robusto/funcional tanto em não dar alarmes falsos (precisão) quanto em não deixar passar defeitos reais (recall) para todas as classes de defeitos simultaneamente.

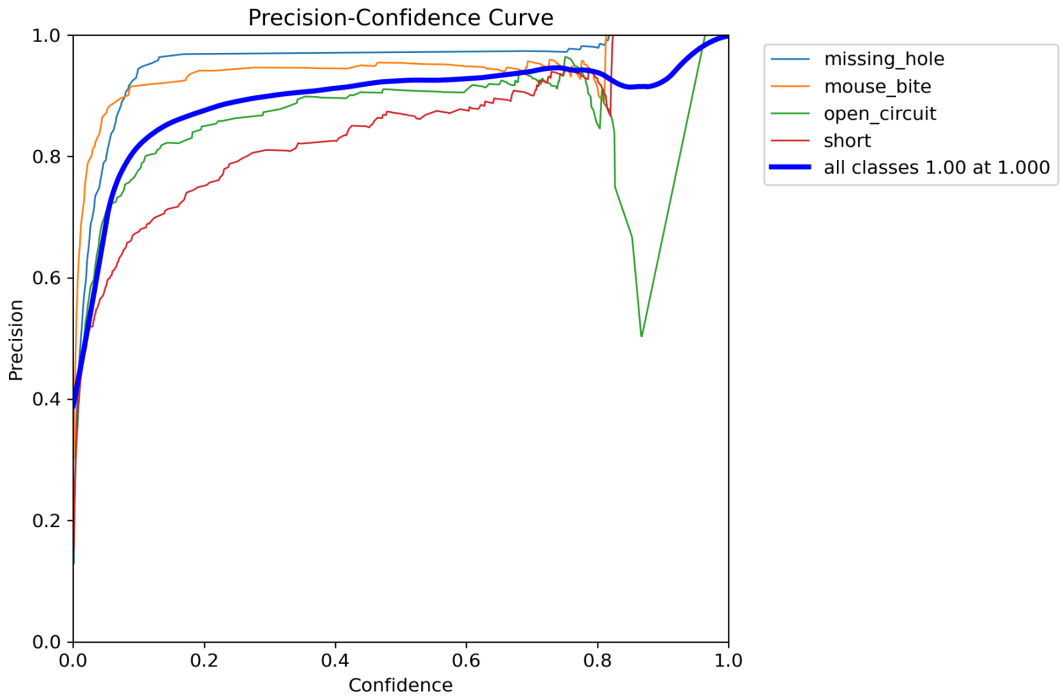
Demonstração dos cálculos (baseado nos dados reais):

Vamos extrair os números brutos da matriz de confusão para demonstrar o cálculo de duas classes distintas: uma com alta performance (missing_hole) e a classe crítica (short).

Exemplo 1: Classe missing_hole (furo faltando), observando a matriz temos:

- VP (Acertos): 185 (Interseção da linha missing_hole com coluna missing_hole).
- FP (Erros do modelo): Olhando a linha missing_hole, vemos o número 5 na coluna background. O modelo achou que tinha furo onde era fundo.
- FN (Defeitos perdidos): Olhando a coluna missing_hole, vemos o número 5 na linha background. O modelo achou que era fundo, mas tinha um furo.
- Precisão: $\frac{VP}{VP+FP} = \frac{185}{185+5} = \frac{185}{190} \approx 0.973$ (97.3%)
- Recall: $\frac{VP}{VP+FN} = \frac{185}{185+5} = \frac{185}{190} \approx 0.973$ (97.3%)

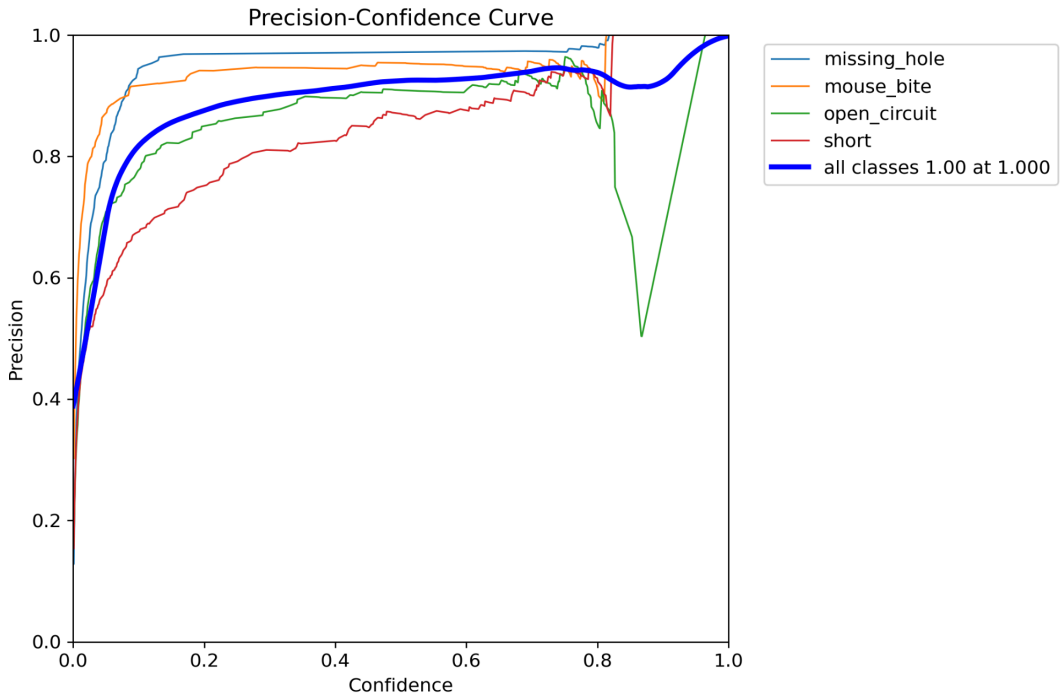
Conclui-se que o modelo é confiável para detectar furos, como confirmado pela curva do gráfico abaixo, onde a linha azul toca o topo.



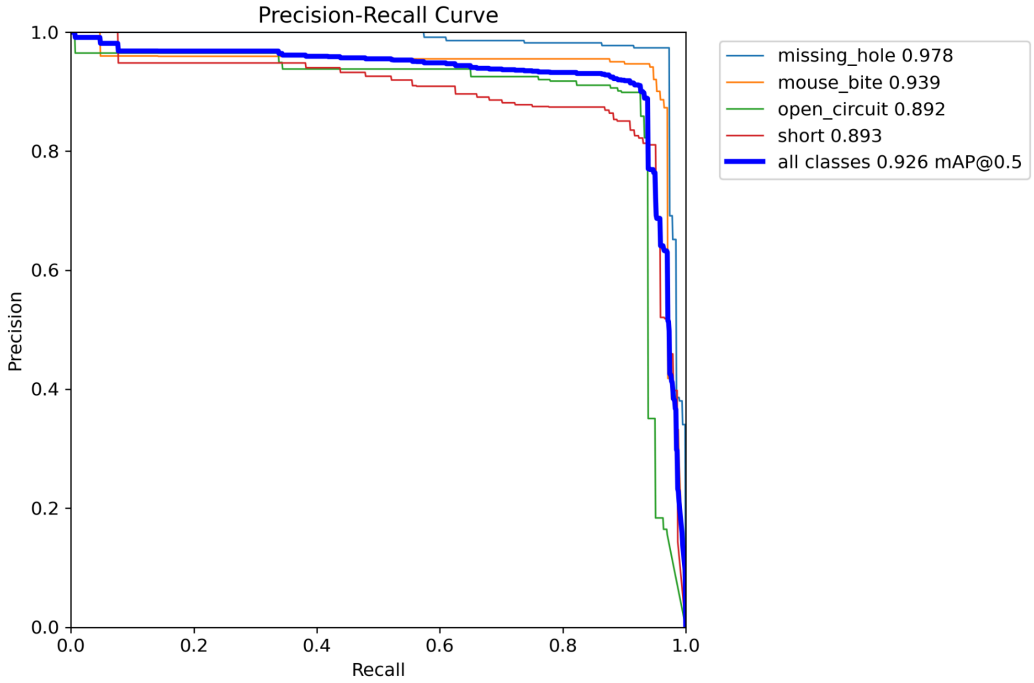
Exemplo 2: Classe short (curto-circuito), observando a matriz temos:

- VP (Acertos): 139 (Interseção linha/coluna short).
- FP (Alarmes falsos): Na linha short, temos o número 34 na coluna background. O modelo "alucinou" curtos-circuitos 34 vezes onde não havia nada.
- FN (Deixou passar): Na coluna short, temos o número 5 na linha background. 5 curtos reais passaram despercebidos.
- Precisão: $\frac{VP}{VP+FP} = \frac{139}{139+34} = \frac{139}{173} \approx 0.803$ (80.3%)
- Recall: $\frac{VP}{VP+FN} = \frac{139}{139+5} = \frac{139}{144} \approx 0.965$ (96.5%)

Conclui-se que o modelo tem um ótimo recall, o que é ótimo para a persona Roberto já que quase nenhum defeito passa. Porém, a precisão é menor (80.3%), indicando que o sistema às vezes para a esteira por engano (falso positivo), confundindo sujeira ou fundo com curto-circuito. Isso explica a queda da linha vermelha no gráfico abaixo:



Agora quanto ao mAP, no gráfico comparativo entre precisão e recall abaixo ele apresenta o valor de 0.926, que é a média da precisão de todas as classes. Quanto mais próximo o valor for de 1.0 (medida da área sob a curva), e quanto mais "gorda" for a curva para o canto superior direito, melhor o modelo. Dessa forma, confirmamos que o experimento foi bem sucedido.



5. Underfitting e Overfitting

5.1 Introdução e Contextualização do Problema

No cenário atual da Indústria 4.0, a automação do controle de qualidade tornou-se um requisito fundamental para a competitividade. Para a empresa **EletroTech**, objeto deste estudo de caso, a inspeção manual de Placas de Circuito Impresso (PCBs) revelou-se um gargalo produtivo insustentável, gerando custos elevados e falhas na detecção de defeitos microscópicos. A aplicação de Visão Computacional, especificamente através da arquitetura YOLOv8 (You Only Look Once), surge como uma solução promissora para automatizar esta tarefa.

No entanto, a implementação de uma rede neural profunda não se resume apenas a alimentar dados num algoritmo. O sucesso de um modelo de Inteligência Artificial depende intrinsecamente da sua capacidade de generalização. Um modelo útil não é aquele que apenas memoriza as imagens que viu durante o treino, mas sim aquele que consegue identificar corretamente padrões (como curtos-circuitos ou mordidas de rato) em imagens inéditas que surgem na linha de produção diária.

Neste contexto, esta atividade técnica foca-se na análise de dois dos maiores desafios no treino de redes neurais: o Underfitting e o Overfitting. O objetivo deste relatório é tratar esses conceitos teóricos e apresentar uma validação experimental, demonstrando como estes fenômenos afetam a confiabilidade do modelo de detecção de falhas em PCBs e documentando as estratégias utilizadas para reproduzir, identificar e mitigar estes comportamentos.

5.2 Fundamentação dos conceitos

Para compreender tecnicamente o Underfitting e o Overfitting, é necessário recorrer ao conceito estatístico do Dilema Viés-Variância. O erro de um modelo preditivo pode ser decomposto em três componentes: viés (bias), variância e erro irredutível. O objetivo do treino é encontrar um equilíbrio ótimo onde o erro total seja minimizado.

5.2.1. Underfitting (Alto Viés)

O Underfitting ocorre quando o modelo estatístico é demasiado simples para capturar a estrutura subjacente dos dados. Em termos técnicos, dizemos que o modelo possui alto viés. Isto significa que a rede neural faz suposições demasiado fortes e simplistas sobre a relação entre a imagem de entrada (pixels da placa) e a saída (classificação do defeito).

- **Efeitos:** O modelo apresenta um desempenho pobre tanto nos dados de treino quanto nos dados de validação. As curvas de perda (*loss function*) estagnam em valores altos prematuramente.
- **Analogia:** Imagine um inspetor novato na EletroTech que mal olhou para o manual de instruções. Ele deixará passar defeitos óbvios (falsos negativos) e marcará placas boas

como defeituosas aleatoriamente, pois não aprendeu as características que definem um "curto-circuito".
- **Causas Comuns:** Arquitetura da rede pouco profunda (poucos parâmetros), tempo de treino insuficiente (poucas épocas) ou características dos dados insuficientes para a complexidade do problema.

5.2.2 Overfitting (Alta Variância)

O Overfitting é o oposto. Ocorre quando o modelo é excessivamente complexo em relação à quantidade e variedade de dados disponíveis, passando a modelar o ruído aleatório do conjunto de treino em vez dos padrões gerais. O modelo possui alta variância, ou seja, é extremamente sensível a pequenas flutuações nos dados de treino.

- **Efeitos:** O modelo apresenta um desempenho excepcional nos dados de treino (erro próximo de zero), mas o desempenho degrada-se significativamente nos dados de validação ou teste. Graficamente, observa-se uma divergência: a curva de perda do treino continua a descer, enquanto a curva de validação começa a subir.
- **Analogia:** Imagine um inspetor que decorou exatamente a posição de cada mancha de pó nas placas que usou para estudar. Se lhe apresentarem uma placa nova, limpa, mas com um defeito numa posição ligeiramente diferente, ele falhará porque "decorou" os exemplos em vez de aprender o conceito de defeito.
- **Consequências na Indústria:** Um modelo em overfitting é perigoso porque dá uma falsa sensação de segurança durante o desenvolvimento, mas falha catastroficamente quando colocado em produção real.

5.3 Cenário A: A Simulação de Underfitting

Neste primeiro experimento, o objetivo era impedir que a rede neural convergisse. Configuramos o hiperparâmetro epochs para apenas 2. Em redes neurais profundas, as primeiras épocas são caracterizadas por ajustes grosseiros dos pesos sinápticos através do algoritmo de *Backpropagation*. Com apenas 2 ciclos completos sobre o dataset, a rede não teve oportunidade matemática de minimizar a função de custo.

Resultados Quantitativos: Os dados extraídos do experimento confirmam a baixa eficiência do modelo:

- **mAP50:** 0.461 (ou **46,1%**).
- **Recall:** 0.314 (ou **31.4%**).
- **Precisão:** 0.753 (ou 75.3%).

O valor do mAP50 indica que, na vasta maioria dos casos, o modelo não conseguiu sequer desenhar uma *Bounding Box* (caixa delimitadora) com uma sobreposição aceitável sobre o defeito real. O Recall de 31.4% é particularmente crítico para a indústria: significa que de cada 100 defeitos reais na linha de produção, o modelo deixou passar 69. No contexto da EletroTech, isto resultaria no envio de milhares de placas defeituosas para os clientes,

destruindo a reputação da empresa. Visualmente, ao inferir nas imagens de teste, o modelo ou não gerou nenhuma detecção, ou gerou caixas aleatórias com graus de confiança baixos. Este é um caso clássico de underfitting, o modelo não aprendeu nada significativo.

5.4 Cenário B: A Robustez Inesperada do YOLO v8 (500 Épocas)

Na tentativa de demonstrar o Overfitting, configuramos o treino para 500 épocas, uma quantidade exagerada para um dataset deste tamanho. Na etapa de testes utilizamos um dataset modificado manualmente pela equipe que visou expor o modelo a estilos diferentes de open_circuit e mouse_bite. A hipótese inicial era que, por volta da época 300 ou 400, a perda de validação começasse a subir, indicando que a rede estava a memorizar o ruído.

Resultados Observados: Contra a intuição inicial, o modelo não entrou em declínio.

- As métricas de precisão e recall estabilizaram num patamar alto.
- Não houve divergência significativa entre as curvas de treino e validação (*Validation Loss* não explodiu).

Por que é que o Overfitting não ocorreu da forma clássica? A resposta reside na arquitetura avançada do YOLO v8. Modelos modernos implementam diversas técnicas de regularização implícitas e explícitas que combatem o overfitting, mesmo quando o utilizador tenta forçá-lo:

1. **Batch Normalization:** A normalização dos lotes ajuda a estabilizar o aprendizado e tem um leve efeito de regularização.
2. **Arquitetura Profunda:** A capacidade do modelo é tal que ele conseguiu encontrar um mínimo local muito estável.

Embora o modelo não tenha sido degradado, treinar por 500 épocas revelou-se um desperdício de recursos computacionais, pois o ganho de performance após a época 50 foi marginal. Isto demonstra que o "Overfitting" moderno é mais difícil de provocar "por acidente" em frameworks robustos.

5.5 Cenário C: Overfitting Forçado e a "Sabotagem" (Viés de Domínio)

Dado que o modelo resistiu ao excesso de épocas, desenhamos um experimento para provar que a rede neural pode, de fato, decorar características erradas se os dados forem enviesados. Este fenômeno é conhecido como falha de Generalização de Domínio.

Metodologia da "Sabotagem":

1. Criamos uma cópia do dataset de treino e aplicamos um filtro de processamento de imagem para alterar a cor de todas as placas de verde para vermelho.
2. Utilizamos o treinamento que apenas usava placas verdes.

3. O objetivo era fazer a rede associar a característica "Cor Verde" à existência de objetos de interesse, ignorando a forma geométrica dos defeitos (furos, cortes).
4. No teste final, apresentamos ao modelo as imagens sabotadas (**placas vermelhas**).

Resultados Catastróficos: A performance do modelo colapsou completamente quando confrontado com o "mundo real" (placas verdes):

- mAP50: 0.03 (3.0%).
- Recall: 0.067 (6.7%).
- Precisão: 0.024 (2.4%).

Análise Profunda: Este resultado é a prova definitiva de um Overfitting contextual severo. O modelo sofreu um "sobreajuste no domínio de treino". A rede neural aprendeu uma função condicional errada. Em vez de aprender: SE (forma == buraco) ENTÃO (defeito) Ela aprendeu: SE (cor == verde E forma == buraco) ENTÃO (defeito)

Quando a cor mudou para vermelho, a primeira condição da "regra" interna da rede falhou, e consequentemente, ela não ativou nenhum neurônio de detecção.

Este experimento é valioso porque simula um erro comum na indústria: treinar um modelo com dados de laboratório (iluminação perfeita, fundo limpo) e esperar que funcione no chão de fábrica (luz variável, fundos diferentes). O modelo ficou "viciado" nas características específicas do treino (a cor verde) e perdeu totalmente a capacidade de generalizar para o problema real.

5.5 Considerações finais sobre os aspectos técnicos

Foi possível compreender de forma prática e aprofundada os mecanismos de aprendizagem das Redes Neurais.

1. O **Underfitting** (Cenário A) mostrou-nos que sem tempo e iterações suficientes, a complexidade do modelo é inútil; o gradiente descendente precisa de tempo para navegar na superfície de erro até um mínimo aceitável.
2. O **Overfitting** (Cenários B e C) revelou-se um fenómeno complexo. Descobrimos que frameworks modernos como o YOLOv8 são resilientes ao overfitting por pura repetição (épocas), graças a mecanismos internos de regularização. No entanto, demonstramos através do experimento de alteração de cor que o modelo é extremamente vulnerável a vieses nos dados.

Para o projeto da EletroTech, a lição mais importante extraída deste estudo é a importância do tratamento adequado no dataset. O perigo real não é treinar demais, mas sim treinar com dados que não representam a realidade. O experimento das placas vermelhas prova que, se o dataset de treino não tiver a variabilidade correta (diferentes cores de placas, diferentes iluminações), o sistema falhará em produção, independentemente da complexidade da arquitetura YOLO.

Portanto, para garantir um modelo robusto, o foco não deve estar apenas no ajuste de hiperparâmetros ou no número de épocas, mas sim na garantia de que os dados de treino cobrem toda a variância esperada no ambiente real da fábrica, evitando que o modelo decore características espúrias como a cor de fundo.